

BLoC (Business Logic Component) pattern is separation of concerns. The app is divided into three main layers:

Data Layer:

Responsible for handling data. It includes:

Models (data structure)

Web Services (API calls)

Repository (connects data sources together)

Business Logic Layer:

This is where Cubit or BLoC exists.

It acts as a middle layer between the UI and the data.

It receives input and outputs states.

Presentation Layer (UI):

Displays data to the user and reacts to state changes.

Event:

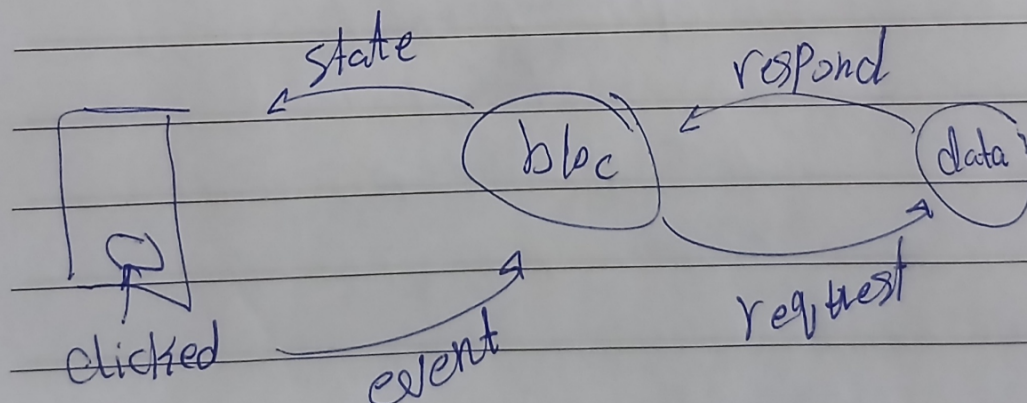
Represents an action triggered by the user.

State:

Represents the UI condition at a specific moment

Bloc: Business Logic Component
design pattern . state management
Architecture Patterns

لایه های مختلف، لایه های مختلف، لایه های مختلف
layers و لایه، لایه



Cubit:

Simpler and easier to use

Does not use events

Directly emits states

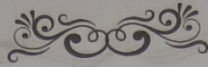
Suitable for small or simple features

BLoC:

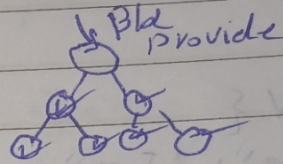
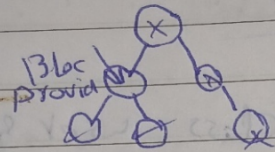
More structured and scalable

Uses both Events and States

Better for large applications



- Bloc Provider $\rightarrow$ bloc فوق widget tree
يشيل bloc لكل widget فيها ، و free



By default, BlocProvider make the code lazy

* UI gotta call it
BlocProvider.of $\leftrightarrow$ (Context)

- BlocBuilder $\rightarrow$ bloc provider ديقو
- rebuild itself w each state

- BlocListener $\rightarrow$ لما لي state change
يعمل action و state change

Bloc Consumer $\rightarrow$ BlocBuilder + BlocListener

Dio:

A powerful library used for making API requests.

JSON to Dart Models:

Converts JSON responses into Dart objects.

Mapping:

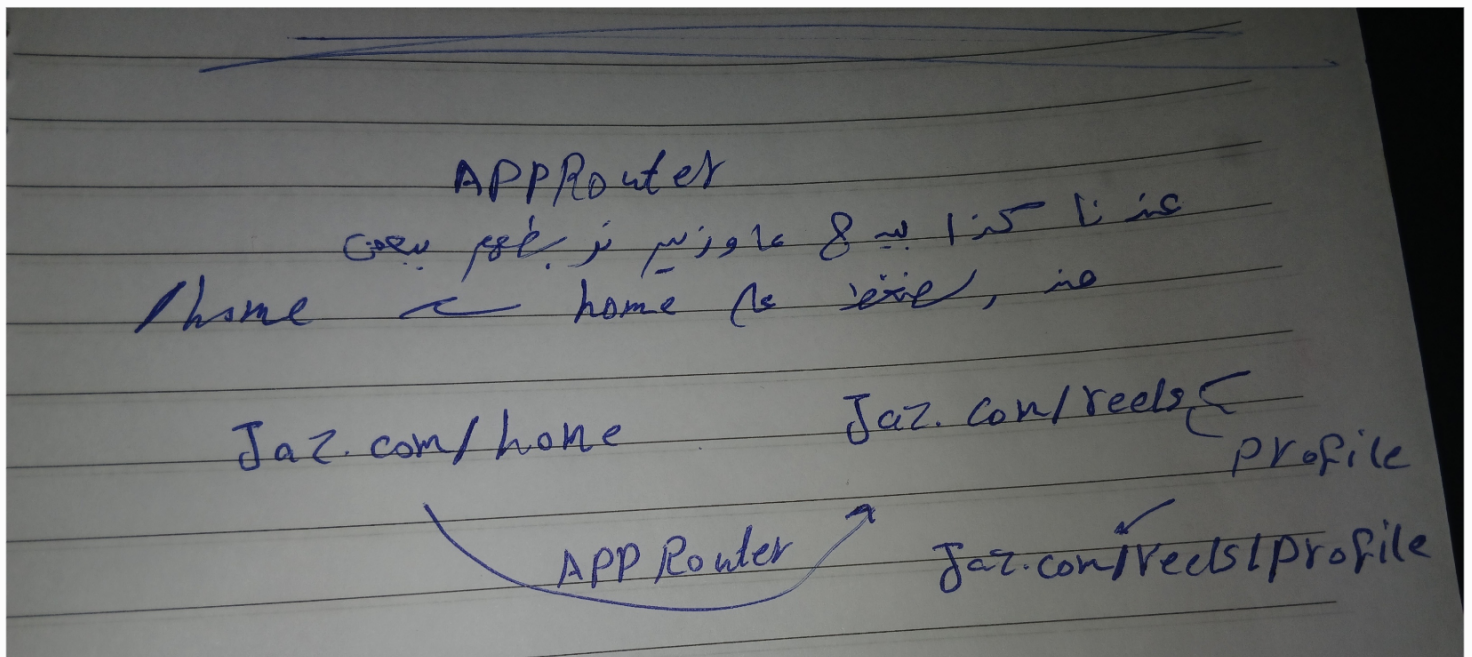
Used to convert a list of JSON objects into a list of models using fromJson.

****Instead of random navigation between screens:**

We use a centralized AppRouter

Define routes using onGenerateRoute

This makes navigation more organized and controlled



The App Router in Flutter makes navigation between screens more organized and flexible. It helps manage routes in a cleaner way, especially in large applications, and makes the app easier to maintain and scale as more features are added.

Bloc provider in detail

بسیار ساده و widget و پرفورمانس و Bloc از، Cubit باقی، widgets الی جوامع
فی، تجربه

BlocProvider (

create: (context) => CounterCubit(),

child: MyHomePage(),)

Create = منای پخش Bloc از Cubit وای
widget جوامع از home و قدرت و بل Bloc ده

* هو کانتاینر object فی مته و امه کرده وای
widget موجوده تغییر قدرت و مته و تغییر ده مته
غیر ما تغییر تبعه بایر، صفحات و ال widgets
من

→ Bloc Builder

ده، جزء، الی بایر، الی ال State
اول ما، حاله تغییر، ال widget الی جوامع
بعد rebuild تلقائی و یقوت باقیم، تجربه

BlocBuilder (context, int) <

builder: (context, state) {

return Text('\$state'); }

* اقیم، حاله state

* کل ما تغییر، ال Text یقوت لو امه

Search to Filter in Flutter means filtering a list based on what the user types in the search bar.

As the user enters text, only the matching items are displayed in the UI.

It's usually done using a TextField, onChanged, and the where() method to update the list dynamically.

```
List<String> names = ["Ahmed", "Ali", "Mona", "Mariam"];

TextField(
  decoration: InputDecoration(labelText: "Search Names"),
  onChanged: (value) {
    filteredNames = names.where((name) {
      return name.toLowerCase().contains(value.toLowerCase());
    }).toList();

    setState(() {});
  },
)
)
```


CustomScrollView

~ a very flexible scroll view

Scrolling widgets: انواع مختلفه از اسکرولینگ ویجت ها

مع بعضی ها می تونیم ترکیب کنیم (مثلاً)

- SliverAppBar - SliverList - SliverGrid

بدل ما می تونیم ListView عادی و ظاهرش اینست
هنا می تونیم در یک جزو اسکرول بشود و افقی

* افکره، این اسکرولر اینه می تونیم بنظم اسکرولر
و ده می تونه اسم کل جزو اسکرولر بگیرد
as Section مستقل بقیه اسکرولر، یعنی اینست، او می تونه به هر رتبه می تونه

CustomScrollView is more powerful when you need:

- a dynamic AppBar
- combining Grid and List in the same scroll
- custom scrolling effects
- multiple different sections within one scroll view

A very simple ex:

```
CustomScrollView(  
  slivers: [  
  

```

```
    SliverAppBar(  
      pinned: true,  
      expandedHeight: 200,  
      flexibleSpace: FlexibleSpaceBar(  
        title: Text("Profile"),  
      ),  
    ),  
  ],  
),
```

```
    SliverList(  
      delegate: SliverChildBuilderDelegate(  
        (context, index) {  
          return ListTile(  
            title: Text("Item $index"),  
          );  
        },  
        childCount: 20, ),  
    ),  
  ),  
),
```

animated text:

text that changes or moves with animation instead of appearing static. It can be used to make UI more engaging, like typing effects, fading text, or bouncing words.

You usually implement it using widgets like:

`AnimatedDefaultTextStyle` (for style changes like color, size, weight)

`TweenAnimationBuilder` (for custom animated transitions)

packages like `animated_text_kit` for ready-made effects (typing, fading, rotating text)

```
AnimatedDefaultTextStyle(  
  duration: Duration(seconds: 1),  
  style: TextStyle(fontSize: 30, color:  
Colors.blue),  
  child: Text("Hello Flutter"),  
)
```

وبارك الله فيما رزق.

